



成都东软学院

实 验 报 告

课程名称： 人工智能提示工程

指导教师： 谢心

学 院： 数智应用技术学院

年级专业： 24 级软件工程

班 级： 24402

学 号： 24068249218

姓 名： 杨陆续

实验（1）

【实验目的】

1. 掌握项目文档自动化：学习运用 AI 提示工程技术，自动化生成标准化的项目说明文档（README.md），以准确描述 Python 代码的功能与实验教学目标。
2. 熟悉版本控制基础操作：掌握 Git 的初始配置（包括设置全局用户名与邮箱）、本地仓库初始化及完成首次代码提交的完整 workflow。
3. 开发基础交互式 AI 应用：设计并实现一个可在终端运行的交互式聊天客户端，要求具备流式输出、上下文记忆（自动将历史对话纳入后续交互）及可持续运行直到用户主动退出的能力。

【实验内容】

任务 1：项目初始化与文档生成：在本地创建项目目录，并编写用于生成 README.md 文件的提示词模板或脚本，确保文档包含项目概述、功能列表、使用方法和教学目标。

任务 2：Git 版本控制实践：在项目根目录执行 Git 初始化命令，完成基础配置并进行首次提交，将生成的 README.md 及后续代码纳入版本管理。

任务 3：基础聊天客户端开发：在 practice01 目录下编写 chat_client.py 程序。该程序需通过 API 调用大型语言模型，实现：接收用户输入的循环对话。

以流式（逐字或分块）方式输出模型回复，提升响应感知。

维护一个全局的对话历史列表，每次请求时将全部历史记录作为上下文发送，实现对话连续性。

设置退出机制（如用户输入“exit”或“quit”），结束程序循环。

1.2 实验过程：

环境准备：创建项目主目录（如 ai_prompt_engineering_labs），在内部建立 practice01 子目录。安装必要的 Python 依赖包（如 openai, requests 等）。

文档生成：设计提示词，要求大模型根据给定的代码结构和功能描述，生成格式正确、内容完整的 README.md 文件。可手动调整和完善生成的内容。

Git 操作：在命令行中，于项目根目录执行命令

代码开发：在 practice01/chat_client.py 中实现核心逻辑：

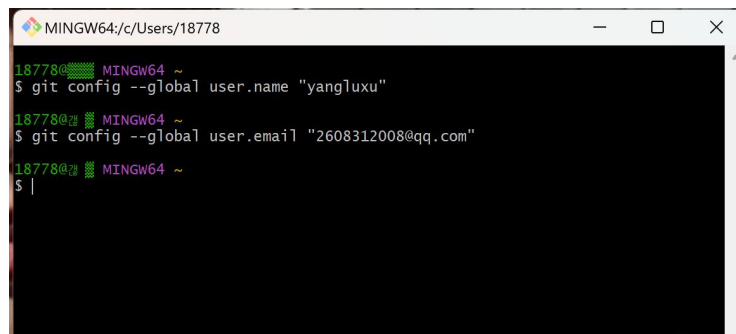
使用 input() 函数获取用户输入。

构建一个 messages 列表，包含系统角色设定、所有历史对话记录以及最新用户输入。

调用大模型 API 时，启用 stream=True 参数以实现流式响应。

在一个 while True 循环中封装整个对话流程，并设定退出条件。

1.2 实验结果：



```
MINGW64/c/Users/18778
18778@MINGW64 ~
$ git config --global user.name "yangluxu"
18778@MINGW64 ~
$ git config --global user.email "2608312008@qq.com"
18778@MINGW64 ~
$ |
```

【实验总结】

本次实验成功构建了一个基础的 AI 对话应用原型。通过实践，我掌握了利用 AI 辅助生成技术文档的方法，巩固了 Git 的基本 workflow，并实现了具备上下文记忆和流畅交互体验的聊天程序。关键在于如何有效构建和动态维护对话消息列表。这为后续实验增加了工具调用等高级功能打下了坚实基础。

实验（2）

【实验目的】

1. 掌握 AI 工具调用（Function Calling）的核心机制：学习如何定义工具（函数）的描述，并将其集成到 AI 助手的系统提示中，使模型能够理解并“决定”调用本地功能。
2. 实现文件系统操作工具集：开发五个具体的本地文件操作函数，为 AI 助手扩展对本地文件系统的控制能力。
3. 升级聊天客户端：基于实验一的程序，开发能支持工具调用的增强版聊天客户端，实现“自然语言指令 -> AI 决策 -> 工具执行 -> 结果返回”的自动化流程。

【实验内容】

新建目录与基础工具开发：在项目下新建 `practice02` 目录，并在其中编写包含五个文件操作函数的 Python 模块（如 `file_tools.py`）：

```
list_files(directory)

rename_file(directory, old_name, new_name)

delete_file(directory, filename)

create_file(directory, filename, content)

read_file(directory, filename)
```

工具调用集成：在 `practice02` 目录下，基于 `practice01/chat_client.py` 开发新的 `tool_chat_client.py`。核心任务包括：

将上述五个函数的描述（名称、功能说明、参数 JSON Schema）按照大模型要求的格式（如 OpenAI 的 `tools` 参数格式）进行定义。

将该工具定义列表作为系统提示的一部分，或在与模型的交互请求中携带。

修改消息处理逻辑：在接收到模型的回复后，解析其中是否包含工具调用请求。若有，则自动执行对应的本地函数，并将执行结果以特定格式（如新的 `tool_call` 消息）追加到对话历史中，再次发送给模型以获得面向用户的最终回答。

2.1 实验过程：

工具函数实现：在 `practice02/file_tools.py` 中，使用 Python 的 `os`、`shutil` 等标准库，安全地实现五个文件操作函数，并做好错误处理。

工具定义：在 `tool_chat_client.py` 中，创建一个 `tools` 列表，其中每个元素都是一个字典，详细描述一个工具。

客户端逻辑改造：

在调用大模型 API 时，将 tools 参数传入。

在 while 循环中，增加对 API 返回响应的判断：如果响应中包含 tool_calls，则依次执行每个被调用的工具，收集结果。

将工具执行结果封装成{"role": "tool", "content": result, "tool_call_id": id}格式的消息，追加到历史记录。

携带包含工具执行结果的新历史记录，再次请求大模型，让其生成面向用户的自然语言回答。

2.2 实验结果：

```
PS D:\tmp> & d:/tmp/venv/Scripts/python.exe d:/tmp/practice02/chat_client.py
=== LLM 聊天客户端（LM Studio 适配版）===
按 Ctrl+C 退出
=====

你：我叫蒋丞
助手：

你好呀，蒋丞！很高兴认识你。 🤖

“蒋”姓很有底蕴，“丞”字也很雅致，听起来是个很特别的名字呢。

今天有什么想聊的话题，或者需要我帮忙处理的事情吗？无论是学习、工作、创作，还是单纯想找个人说说话，随时都可以告诉我～ 😊
你：我叫什么名字
助手：

你叫**蒋丞**。
```

这是你刚才告诉我的名字呀，我记着呢！这个名字听起来很有气质呢～ 😊

你：█

【实验总结】

实验二是一次从“纯对话”到“具备行动能力”AI 的关键升级。通过本次实验，我深入理解了 AI 工具调用的工作范式。关键在于精确的工具描述和客户端对工具调用响应的解析与执行逻辑。这使 AI 从信息提供者转变为任务执行者，极大地扩展了其应用边界。在实现

中，需特别注意文件操作的安全性和错误处理，避免产生非授权访问或数据丢失。

实验（3）

【实验目的】

1. 扩展 AI 助手的能力边界：在已具备本地文件操作能力的基础上，为 AI 助手新增网络访问能力，使其能够获取外部信息。
2. 实践工具集的动态增补：学习如何在现有工具调用框架中，无缝集成新的功能模块。
3. 维护项目文档的完整性：随着项目功能迭代，及时更新项目总览文档，保持开发与文档的同步。

【实验内容】

理解项目架构：回顾并分析实验一、二形成的项目代码结构，特别是工具定义、注册和调用流程，确保新增功能能够平滑集成。

开发网络访问工具：在 `practice02` 目录下，新增一个网络访问功能函数，例如 `fetch_url(url)`。该函数应能通过 HTTP 请求（可使用 `requests` 库或调用 `curl` 命令）获取指定 URL 的网页内容，并将文本或状态信息返回。

集成新工具：在 `practice02/tool_chat_client.py` 中，将 `fetch_url` 函数的描述添加到 `tools` 工具定义列表中，更新系统提示或相关配置，使大模型知晓此新能力。

更新项目文档：根据新增的 `fetch_url` 功能，更新项目根目录下的 `README.md` 文件，在功能列表、使用示例等部分补充对网络访问能力的说明。

3.1 实验过程：

代码审查：通读 `practice02` 目录下的 `file_tools.py` 和 `tool_chat_client.py`，明确工具函数的接口格式和客户端的调用逻辑。

实现网络工具：

安装 `requests` 库：`pip install requests`。

编写 `fetch_url(url)` 函数，使用 `requests.get(url)` 发送请求，处理可能的异常（如超时、404 错误），并返回网页的文本内容或错误信息。

客户端更新：

在 `tool_chat_client.py` 中导入新的 `fetch_url` 函数。

在 `tools` 列表末尾添加该工具的定义字典。

（无需修改核心调用逻辑，因为实验二的框架已支持动态执行 `tools` 列表中定义的任何函数）。

文档更新：编辑 `README.md`，在“功能特性”部分增加“网络访问：可查询指定 URL 的网页内容”，并提供一个示例对话。

3.2 实验结果：


```
PS D:\tmp> & d:/tmp/venv/Scripts/python.exe d:/tmp/practice02/tool_chat_client.py
=== LLM 聊天客户端（支持工具调用） ===
按 Ctrl+C 退出聊天
=====
成功加载 .env 文件：d:\tmp\.env
API URL: http://localhost:1234/v1
模型名称：Qwen3.5-4b

你：通过访问http://wttr.in/青城山，帮我查看明天的天气
助手：
```

```
10 km | \n | 0.0 mm
| 0% | <span class="ef111"> ' ' ' ' </span> 0.0 m
m | 83% | <span class="ef111"> ' ' ' ' </span> 0.0
mm | 83% | <span class="ef111"> ' ' ' ' </span> 0
.1 mm | 100% | \n |
|
| \n
| \n
| 4月16日星
期四 |
| \n | 早上 | 傍晚 | 中午
| \n | 夜间 | \n |
```

【实验总结】

实验三是对工具调用框架扩展性的成功验证。通过新增一个 `fetch_url` 网络工具，我们以极低的成本（主要工作是编写一个函数及其描述）为 AI 助手赋予了获取实时外部信息的能力。这充分展示了基于工具调用构建的 AI 应用的优势：能力模块化、易于扩展。同时，本次实验也强调了文档维护的重要性，完备的文档是项目可持续协作和发展的基础。整个实验系列（从基础对话、到本地操作、再到网络访问）清晰地展示了一个功能不断增强的 AI 应用代理（AI Agent）的开发路径。